

SentinelPath SA

Disaster Response Navigator

Java 17 + JavaFX | Dijkstra & A*

Computer Science — Pathfinding & Image Processing

Team Members

Student Number	Initials	Surname
222151999	M	Kosa
223025836	N	Chingosho
222119329	SS	Hleza

Background

South Africa faces recurring localised disasters — from KZN floods to informal settlement fires.

Emergency responders need rapid, intelligent routing support when standard maps fail.

South Africa's National Disaster Management Framework identifies infrastructure failure as the primary barrier to effective emergency response ? yet no lightweight tool exists that converts live aerial imagery into a traversable risk graph without pre-labelled geospatial data (DFFE eGIS, 2023; JBA Risk Management, 2023). SentinelPath SA directly addresses this gap by enabling responders to generate a navigable weighted graph from any aerial photograph in seconds.

■ SA Context

South Africa's geography and infrastructure create unique disaster-response challenges.

Informal settlements, flood plains, and damaged roads complicate navigation.

■ The Gap

Existing navigation tools assume intact roads and ignore terrain traversal costs.

No existing lightweight tool maps aerial imagery to a risk-weighted graph in real time.

■ Our Goal

SentinelPath SA converts any aerial or satellite image into a navigable weighted graph.

Responders get an optimal, risk-aware route at the click of a button.

The Problem

During disasters, responders must navigate complex, changing terrain — fast.
Standard maps don't account for terrain risk or real-time blockages.

■ Terrain Blindness

No consideration of road vs water vs vegetation traversal cost.

All cells are treated equally — a road and a flooded field look the same to the navigator.

This can send vehicles into impassable terrain.

■ Dynamic Hazards

Floods, fires, and debris change the optimal route in real time.

Static maps cannot reflect live blockages that emergency crews encounter on the ground.

A route that was clear minutes ago may now be cut off.

■ No Comparison

Responders can't see trade-offs between different routing strategies.

Without algorithm comparison, teams can't know if a shorter path carries higher risk.

Informed decisions require side-by-side data, not a single opaque recommendation.

Proposed Solution

SentinelPath SA converts aerial imagery into a risk-weighted graph and finds the safest route using Dijkstra's Algorithm & A*, with full CRUD graph interaction in a JavaFX GUI.

Running Dijkstra and A* simultaneously allows responders to compare an exhaustive, guaranteed-optimal route against A*'s heuristically guided path — typically 2-3x faster — so time-critical decisions reflect the speed-coverage trade-off (Hart et al., 1968; Dijkstra, 1959). Every graph mutation triggers instant re-computation, ensuring the displayed route always mirrors live terrain.

1

Load Aerial Image

Any satellite or drone photograph (.jpg / .png) is loaded as the map canvas.

2

Classify Terrain

A 50×50 grid is overlaid. Each cell's average RGB is matched via Euclidean distance to 5 terrain types: Road, Grass, Vegetation, Water, Blocked.

3

Build Weighted Graph

2,500 nodes connected by up to 8 directional edges. Edge weights derived from terrain classification (Road=1, Water=1000).

4

Find Optimal Path

Dijkstra & A* run simultaneously. Results — cost, nodes explored, time — displayed side-by-side for comparison.

Result: Interactive map with colour-coded terrain, animated path, and side-by-side algorithm stats

System Architecture

How It Works

1. Image divided into 50×50 grid cells
2. Average RGB colour sampled per cell
3. Euclidean distance to 5 reference colours
4. Nearest colour → terrain type assigned
5. Blocked cells excluded from graph



Road

Cost: 1



Grass

Cost: 5



Vegetation

Cost: 10



Water

Cost: 1000



Blocked

Impassable

Terrain Classification

How It Works

1. Image divided into 50×50 grid cells
2. Average RGB colour sampled per cell
3. Euclidean distance to 5 reference colours
4. Nearest colour → terrain type assigned
5. Blocked cells excluded from graph



Road

Cost: 1



Grass

Cost: 5



Vegetation

Cost: 10



Water

Cost: 1000



Blocked

Impassable

Pathfinding Algorithms



Dijkstra's Algorithm

$$f(n) = g(n)$$

- ✓ Explores all nodes by cumulative cost
- ✓ No directional bias — exhaustive frontier
- ✓ Guaranteed optimal path
- ✓ Custom binary min-heap priority queue
- ✓ Early exit when destination dequeued

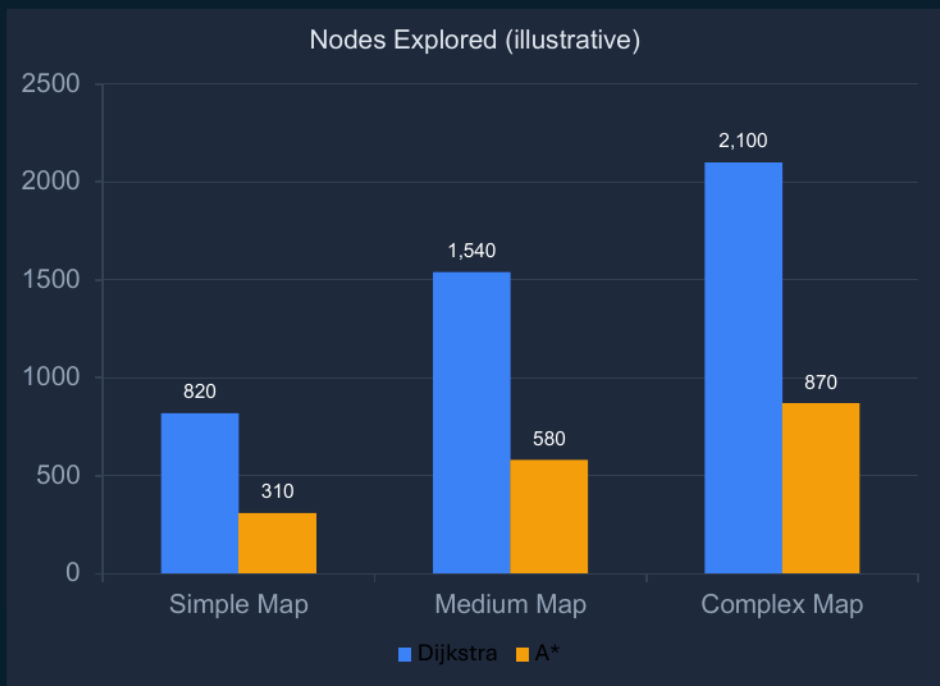


A* Algorithm

$$f(n) = g(n) + h(n)$$

- ⚡ $h(n) = \sqrt{(dr^2 + dc^2)} \times 10$ (Euclidean)
- ⚡ Directs search toward the target
- ⚡ Admissible: $h(n)$ never over-estimates
- ⚡ Explores significantly fewer nodes
- ⚡ Same optimal result, better performance

Algorithm Comparison



Metric	Dijkstra	A*
Heuristic	None	Euclidean × 10
Optimal?	Yes	Yes
Nodes (simple)	~820	~310
Nodes (complex)	~2100	~870
Speed (typical)	Baseline	2–3× faster
Use case	Verification	Live response

Key Features



Zoom & Pan

Scroll to zoom 0.5x–6.0x
Click-drag to pan around the grid



Mini-Map

Persistent 130×130 thumbnail
always shows full grid + path



Simulate Disaster

Random flood clusters
blocking terrain cells dynamically



Edge Management

Delete or restore specific
edges to simulate road closures



Live Comparison

Both algorithms run every time
Shows cost, nodes, and ms side-by-side



Hover Inspection

Hover any node to see
terrain type + coordinates

Future Work

01

Multi-Agent Routing

Independent optimal paths for multiple responder teams simultaneously

02

Live Satellite Data

GIS / satellite feed integration for real-time terrain updates

03

GeoJSON / KML Export

Export computed routes to field navigation devices

04

ML Terrain Classifier

Deep learning model for more accurate terrain recognition

05

Web / Mobile Deployment

REST API exposing pathfinding, with a mobile-friendly front-end

06

Bidirectional A* / JPS

Jump Point Search for faster performance on large-scale grids

References

[1]

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. DOI: 10.1007/BF01386390

[2]

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2), 100–107.

[3]

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. [Weighted graphs & priority queues — Ch. 22–24]

[4]

Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson. [RGB colour-space distance metrics — Ch. 10]

[5]

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley. [Adaptable priority queues — Ch. 14]

[6]

South African Weather Service (SAWS). (2022). *KwaZulu-Natal Catastrophic Flooding Event — Post-Event Report*. Pretoria: SAWS. Available: <https://www.weathersa.co.za>

[7]

Department of Forestry, Fisheries & the Environment (DFFE). (2023). *Environmental Geographic Information System (eGIS)*. Available: <https://www.dffe.gov.za/egis>

[8]

JBA Risk Management. (2023). *Flood Map Insight for South Africa*. Available: <https://www.jbarisk.com/knowledge-hub/insights/flood-map-insight-for-south-africa/>

[9]

Neis, P., & Zielstra, D. (2014). Recent developments in volunteered geographic information research: The case of OpenStreetMap. *Future Internet*, 6(1), 76–106. DOI: 10.3390/fi6010076

[10]

Florczyk, A. J., et al. (2019). *Description of the GHS Urban Centre Database 2015*. European Commission JRC. DOI: 10.2760/037310 [Satellite imagery classification for urban risk mapping]